LIBRARY MANAGEMENT SYSTEM

Group: Group 01	
Group Members	**<Member 1 Name> - <SE20xxx1> <Member 2 Name> - <SE20xxx2> <Member 3 Name> - <SE20xxx3> <Member 4 Name> - <SE20xxx4>**
Lecturer	**<Lecturer Name>**
Class	**<SE20xx>**

#	Student ID	Full Name	Completion	Tasks Performed
1	*SE20xxx1*	*Member 1 (Leader)*	*100%*	***User, Member, IMemberDAO, MemberDAO, MemberManagement, addMember, updateMember, deleteMember, filterByTier, Members.txt***
2	*SE20xxx2*	*Member 2*	*100%*	***Book, TextBook, ReferenceBook, IBookDAO, BookDAO, BookManagement, calculatePrice, rateBook, recommendSimilarBooks, reserveBook, Books.txt***
3	*SE20xxx3*	*Member 3*	*100%*	***BorrowRecord, IBorrowDAO, BorrowDAO, BorrowManagement, borrowBooks, returnBooks, extendLoan, reportLostOrDamaged, exportReceiptToFile***
4	*SE20xxx4*	*Member 4*	*100%*	***DataInput, DataValidation, Menu, printAsciiBarChart, FileManager, Program, displaySystemDashboard, exportSystemSummaryReport***

Milestone 1(Lab01 - Week 3) Template — Requirements & Initial Class Diagram

Document Title: Milestone 1 – Requirements Analysis, Decomposition & Abstraction

1. Project Title

2. Problem Description (5–7 sentences)

"In this project, I aim to design a system that manages multi-category books, library members, and borrowing/returning transactions in a university library. The main users will be librarians and library administrators who oversee daily catalog updates, reader registration, and book circulation. The system will help solve the problem of manual record-keeping errors, difficulty in tracking overdue loans, inaccurate book stock counts, and lack of member credit evaluation. The key functions include book inventory management (TextBook and ReferenceBook), member management with automated membership tiers (SILVER, GOLD, VIP) and reputation points, loan processing with due-date extension and lost/damaged book compensation, star ratings with smart book recommendations, FIFO reservation queues, and console ASCII bar charts. This project will apply OOP and Computational Thinking to decompose the problem and build an efficient solution."

3. Functional Requirements

- **Book Management:** Add, edit, delete, and search books (TextBook and ReferenceBook by ID, keyword, price range, or category); sort books by calculated price or publish year; rate books (1–5 stars); recommend top 3 similar books; and reserve out-of-stock books via a FIFO waiting queue.
- **Member Management:** Add, edit, delete, search, and sort library members; automatically classify membership tiers (VIP ≥ 150 pts, GOLD 100–149 pts, SILVER < 100 pts) to apply borrowing limits (10 / 7 / 5 books) and fee discounts (25% / 10% / 0%), and auto-lock cards when reputation drops below 0.
- **Borrow & Return Management:** Create borrow tickets with stock deduction; process book returns with stock restoration, overdue fine calculation (5,000 VND/day/book), and automatic reservation notification; extend active loans (+7 days, max 1 time); penalize lost/damaged books (150% of polymorphic book price); and export transaction receipts (Receipt_<ID>.txt).
- **System Integration, Visual Analytics & File I/O:** Display console ASCII bar charts (Inventory, Member Tiers, Top Borrowed Books), synchronize UTF-8 text files (Books.txt, Members.txt, BorrowRecords.txt), and export Library_Summary_Report.txt.

4. System Decomposition (CT: Decomposition)

"The system can be divided into the following modules:

1. **Core.Entities Module:** Models domain entities (User, Member, Book, TextBook, ReferenceBook, BorrowRecord) with encapsulated states and polymorphic behaviors.
2. **DataObjects.DAO Module:** Manages ArrayList collections and UTF-8 file persistence through standardized interfaces (IBookDAO, IMemberDAO, IBorrowDAO, BookDAO, MemberDAO, BorrowDAO, FileManager).
3. **BusinessObject.UI Module:** Handles interactive console menus and tabular displays (Program, BookManagement, MemberManagement, BorrowManagement).
4. **Utilities Module:** Provides reusable input handling, regex validation, and ASCII bar chart rendering (DataInput, DataValidation, Menu).

Each module has a clear responsibility in the system."

5. Identified Classes (CT: Abstraction)

Class Name	Attributes	Methods

User (abstract)	# id: String # name: String # phone: String # email: String	+ getId() / setId(String) + getName() / setName(String) + getPhone() / setPhone(String) + getEmail() / setEmail(String) + displayInfo(): void (abstract)
Member	- membershipDate: String - status: String - reputationPoints: int	+ getMembershipTier(): String + getTierDiscountRate(): double + getMaxBorrowLimit(): int + adjustReputationPoints(int): void + displayInfo(): void + toFileString(): String
Book (abstract)	# bookId, title, author, publisher: String # publishYear, quantity, ratingCount: int # basePrice, ratingSum: double # reservationQueue: List<String>	+ calculatePrice(): double (abstract) + displayInfo(): void (abstract) + toFileString(): String (abstract) + addRating(int): void + getAverageRating(): double + addReservation(String): boolean + popNextReservation(): String
TextBook	- status: String ("New"/"Old") - subject: String	+ calculatePrice(): double + displayInfo(): void + toFileString(): String
ReferenceBook	- tax: double - field: String	+ calculatePrice(): double + displayInfo(): void + toFileString(): String
BorrowRecord	- recordId: String - member: Member - borrowedBooks: List<Book> - borrowDate, dueDate, returnDate: LocalDate - status: String - extensionCount: int - compensationFee: double	+ calculateFine(): double + calculateTotalBorrowFee(): double + calculateDefaultCompensationFee(): double + extendDueDate(int): boolean + isOverdue(): boolean + displayInfo(): void + toFileString(): String

6. Relationships between real-world objects

- **Inheritance (Is-A):** Member extends User; TextBook and ReferenceBook extend Book.
- **Aggregation (Has-A):** BorrowRecord aggregates 1 Member and a List<Book> of borrowed books.

7. Encapsulation Evidence (Code Snippets)

```
public class Member extends User {
    // 1. Private attributes
    private String membershipDate;
    private String status;           // "Active" or "Inactive"
    private int reputationPoints;    // Default: 100 points (GOLD tier)
```

```
    // 2. Constructor
    public Member(String id, String name, String phone, String email,
                  String membershipDate, String status, int reputationPoints) {
        super(id, name, phone, email);
        this.membershipDate = membershipDate;
        this.status = status;
        this.reputationPoints = reputationPoints;
    }

    // 3. Getter/Setter & encapsulated state rule
    public int getReputationPoints() { return reputationPoints; }
    public void setReputationPoints(int reputationPoints) { this.reputationPoints =
reputationPoints; }
    public void adjustReputationPoints(int delta) {
        this.reputationPoints += delta;
        if (this.reputationPoints < 0) this.status = "Inactive";
    }
}
```

8. Reflection (4–6 sentences)

"In this milestone, I learned that decomposition helps me break down complex tasks into manageable parts such as separating domain entities, data access objects, user interfaces, and validation utilities. I also practiced abstraction by identifying the essential classes needed in the system (User, Member, Book, TextBook, ReferenceBook, and BorrowRecord) while omitting unnecessary physical details. Describing the Is-A inheritance relationships and Has-A aggregation links between BorrowRecord, Member, and Book clarified how real-world library transactions work. This analysis will guide the next milestone where I implement encapsulation and ArrayList collection managers."

Milestone 2(Lab 02 - Week 5) Template — Encapsulation & Basic Oparators

Document Title: Milestone 2 – Pattern Recognition

1. Summary of Progress (3–5 sentences)

"In this milestone, most objects require Create, Read (display information), Update, and Delete (CRUD) operations and search information. As a result, the project uses ArrayList<Book>, ArrayList<Member>, and ArrayList<BorrowRecord> inside DAO classes to manage collections of objects dynamically. Furthermore, input validation and exception handling were implemented in DataInput and DataValidation to ensure that all objects remain valid throughout their lifecycle."

2. Implemented management Classes Overview

Name class: *BookDAO (implements IBookDAO)*

Description: *The BookDAO class is implemented in the project to manage the collection of books (List<Book>) in the library. This class provides basic operations such as adding new books, updating book information, deleting books, searching for books by ID/keyword/price range/type, sorting, rating stars, recommending similar books, and managing the FIFO reservation queue.*

Basic operator specifications:
Add a new book:
Begin
Step 1: The user inputs the information of the book such as: bookId, title, author, publisher, publishYear, basePrice, quantity, and subclass fields.
Step 2: Validate inputted data (check regex ^B\d{3,}$, verify duplicate ID via findById(id) == null, and check positive numbers).
Step 3: Instantiate TextBook or ReferenceBook, add to bookList, and call saveToFile() to update Books.txt.
End

Name class: *MemberDAO (implements IMemberDAO) & BorrowDAO (implements IBorrowDAO)*

Description: *MemberDAO manages List<Member> (add, update, delete, search, sort, filter by VIP/GOLD/SILVER tier). BorrowDAO manages List<BorrowRecord> (borrowBooks, returnBooks, extendLoan, reportLostOrDamaged, exportReceiptToFile).*

3. Encapsulation Evidence (Code Snippets)

```
public class BookDAO implements IBookDAO {
    private List<Book> bookList;
    private final String FILE_NAME = "Books.txt";

    @Override
    public boolean addBook(Book book) {
        if (book == null || findById(book.getBookId()) != null) return false;
        bookList.add(book);
        saveToFile();
        return true;
    }
}
```

4. Explanation of Abstraction / Pattern Recognition

"I noticed that several classes share common method patterns such as add(), update(), delete(), findById(), getAll(), loadFromFile(), and saveToFile(). Based on these patterns, I designed my methods more consistently through the IBookDAO, IMemberDAO, and IBorrowDAO interfaces. This helped improve the clarity and reusability of the code."

5. Challenges and Solutions

"One challenge I faced was handling invalid console inputs (such as entering text into numeric prompts) and preventing invalid negative quantities inside entities. I solved it by encapsulating all Scanner operations inside Utilities.DataInput with try-catch loops and adding guard conditions in entity setters. This helped me better understand encapsulation."

6. Reflection (4–6 sentences)

"In Milestone 2, implementing encapsulation and pattern recognition taught me how to protect internal ArrayList data structures from unauthorized external modification. By exposing only well-defined CRUD and search methods through DAO interfaces, we ensured that business rules such as unique IDs and non-negative stock quantities are never bypassed. Writing step-by-step pseudocode before coding complex workflows like book borrowing and smart recommendations also helped our group avoid logical bugs. Finally, recognizing recurring CRUD patterns allowed all four members to build consistent management classes."

Milestone 3 (Lab03 - Week 7) Template — Inheritance & Polymorphism

Document Title: Milestone 3 - Hierarchy & Polymorphism,
- Generalization (Advanced Abstraction)

1. Summary of New Additions

"In this milestone, I created a class hierarchy using inheritance (User → Member and Book → TextBook, ReferenceBook) and implemented polymorphism by overriding methods (calculatePrice(), displayInfo(), and toFileString()). This helps the system handle different types of objects using a common interface."

2. Class Hierarchy Design (CT: Generalization)

"The parent class is **Book** (abstract superclass for publications) and **User** (abstract superclass for users).
The child classes are **TextBook, ReferenceBook** (extending Book) and **Member** (extending User).
The shared attributes include **bookId, title, author, publisher, publishYear, basePrice, quantity, ratingSum, ratingCount, reservationQueue** in Book, and **id, name, phone, email** in User.
Each subclass adds unique behaviors such as **TextBook calculating price by status (New = 100%, Old = 50% of basePrice), ReferenceBook calculating price with import tax and a 15,000 VND hardcover fee, and Member computing VIP/GOLD/SILVER tiers and discounts**."

3. UML Hierarchy Diagram

(Refer to the UML Hierarchy Diagram in the LaTeX / Markdown specification showing TextBook --|> Book, ReferenceBook --|> Book, and Member --|> User.)

4. Polymorphism Demonstration (Code Snippet)

```
Book a = new TextBook("B001", "Java OOP", "Nguyen Van A", "NXB GD", 2024, 120000, 15,
"New", "CNTT");

Book b = new ReferenceBook("B006", "Clean Code", "Robert Martin", "Prentice Hall",
2022, 350000, 5, 0.10, "SE");
```

```
a.displayInfo();   // polymorphism: calls TextBook.displayInfo()
b.displayInfo();   // polymorphism: calls ReferenceBook.displayInfo()
double priceA = a.calculatePrice(); // polymorphism: calls TextBook.calculatePrice()
```

In our project, all books are stored in a single List<Book> collection using upcasting. At runtime, the JVM uses dynamic method dispatch to invoke the overridden calculatePrice(), displayInfo(), and toFileString() methods of the actual subclass (TextBook or ReferenceBook) without requiring instanceof checks.

5. Algorithmic Thinking Evidence

"I designed the overridden methods by analyzing how each subclass behaves differently. For TextBook, calculatePrice() checks if status equals 'New' (100% basePrice) or 'Old' (50% basePrice). For ReferenceBook, calculatePrice() computes basePrice + (basePrice * tax) + 15,000 VND. Furthermore, BorrowRecord uses these polymorphic prices to calculate 10% borrowing fees (with VIP/GOLD tier discounts) and 150% lost-book compensation fees. This required algorithmic thinking to ensure that each version of the method follows the correct sequence of steps."

6. Reflection (4–6 sentences)

"In Milestone 3, I learned how inheritance and polymorphism make object-oriented systems extensible and clean. By grouping common attributes into the abstract Book and User superclasses, we avoided duplicating fields and getters/setters across subclasses. Using runtime polymorphism allowed BookDAO and BorrowDAO to iterate over a single List<Book> and invoke calculatePrice() or displayInfo() dynamically. If we need to add a new category such as Magazine or EBook in the future, we only need to extend Book without modifying existing DAO or transaction code."

Milestone 4(Lab04 - Week 8) Template — Final Submission, Evaluation & Reflection

Document Title: Milestone 4 - File I/O
- Final System Integration & CT Reflection

1. System Overview and Integration

The completed Library Management System integrates 19 classes and interfaces across 4 packages (Core.Entities, DataObjects.DAO, BusinessObject.UI, Utilities). Program.java initializes BookDAO, MemberDAO, and BorrowDAO and connects them to the interactive menus in BookManagement, MemberManagement, and BorrowManagement. FileManager.java integrates UTF-8 file reading and writing into the OOP design so that Books.txt, Members.txt, BorrowRecords.txt, transaction receipts (Receipt_<ID>.txt), and system reports (Library_Summary_Report.txt) are persisted automatically.

2. Updated UML Class Diagram (Final Version)

(Refer to the Final 4-Package UML Class Diagram in the LaTeX / Markdown specification.)

3. OOP Concepts Application Summary

- **Encapsulation:** All entity fields are private/protected; internal ArrayList collections are encapsulated inside BookDAO, MemberDAO, and BorrowDAO with validated getters, setters, and state methods.
- **Inheritance:** Member extends User; TextBook and ReferenceBook extend Book.
- **Polymorphism:** Method overriding of calculatePrice(), displayInfo(), and toFileString() across Book subclasses; interface polymorphism via IBookDAO, IMemberDAO, and IBorrowDAO.
- **Abstraction:** Abstract superclasses User and Book; DAO interfaces abstracting file and list operations from the UI layer.

4. CT Skills Evaluation

Decomposition
Abstraction
Pattern Recognition
Generalization
Algorithmic Thinking Evaluation & Iteration

"I applied decomposition when I separated the system into modules such as Core.Entities, DataObjects.DAO, BusinessObject.UI, and Utilities across our four team members.
I used abstraction by focusing on key attributes such as basePrice, quantity, reputationPoints, dueDate, and reservationQueue while ignoring irrelevant physical details.
I demonstrated pattern recognition in BookDAO, MemberDAO, and BorrowDAO where CRUD, search, and UTF-8 file synchronization follow a unified interface pattern, as well as in Menu.printAsciiBarChart().
I applied generalization by designing the abstract Book and User superclasses to handle multiple subclasses uniformly.
I applied iteration when I reviewed and improved my class diagram and business algorithms across the four milestones to support membership tiers, smart recommendations, FIFO reservations, loan extensions, and 150% lost-book penalties."

5. System Testing & Evidence

```
========================================================================
                BANG TONG HOP TOAN HE THONG (SYSTEM DASHBOARD)
========================================================================
  [1. KHO SACH]     Tong so dau sach: 10 dau sach (217 cuon ton kho)
                    Tong gia tri tai san kho sach: 36,649,000 VND
  [2. DOC GIA]      Tong so the doc gia: 5 doc gia (VIP: 1 | GOLD: 2 | SILVER: 2)
  [3. MUON - TRA]   Tong so phieu muon: 3 phieu (Dang muon: 2 | Qua han: 2)
                    Phi muon: 130,200 VND | Tien phat: 45,000 VND | Den bu: 0 VND
========================================================================
```

We tested the system by running both normal user workflows and boundary/invalid input test cases across all four subsystems. First, we tested malformed IDs, negative prices, and invalid dates to verify that DataInput and DataValidation prevented runtime crashes. Second, we verified polymorphic pricing, VIP/GOLD/SILVER discounts, loan extensions, 150% lost-book compensation, and FIFO reservation notifications upon book return. Finally, we restarted the application to confirm that Books.txt, Members.txt, BorrowRecords.txt, and Library_Summary_Report.txt preserved all UTF-8 data accurately.

6. Self-Evaluation (Strengths & Limitations)

"One strength of my system is its clean 4-layer OOP architecture enriched with seven practical creative features: Membership Tiers & Reputation Points, Star Ratings & Smart Recommendations, FIFO Reservation Queues, Loan Extensions, 150% Lost Book Compensation, Console ASCII Bar Charts, and automated Receipt/Report file exporting.
However, a limitation is that the system currently runs as a single-user console application backed by flat text files (.txt).
In the future, I can improve by migrating to a relational database (SQL Server/JDBC), refactoring traditional loops and anonymous Comparator classes to Java 8+ Lambda Expressions and Stream API, and building a JavaFX GUI."

7. Final Reflection (6–8 sentences)

"Completing the Library Management System across all four milestones significantly strengthened my practical command of Object-Oriented Programming and Computational Thinking. From OOP, I learned how Encapsulation protects domain invariants, how Inheritance eliminates duplicate code, and how Polymorphism and Abstraction decouple high-level business workflows from concrete subclass details. My Computational Thinking skills improved markedly as our team practiced Decomposition to split the project into four independent modules and Pattern Recognition to standardize DAO interfaces and input validators. Designing algorithms for smart book recommendations, overdue fine calculation, reputation tier discounts, and ASCII bar chart scaling also sharpened my algorithmic thinking. Furthermore, integrating UTF-8 File I/O with defensive try-catch blocks taught me how to build resilient software that recovers gracefully from bad inputs or missing files. Next time, I would establish the final file serialization schema and cross-module interface signatures even earlier in Week 1 so that team integration goes even faster."